SISTEMAS OPERACIONAIS

Comunicação e sincronização entre processos e *threads*

Prof. Matheus Alcântara Souza
Eng. Software/Sist. Informação
PUC Minas

SO e cenário atual

> Cenário comum: multiprogramação, tempo compartilhado, paralelismo real, *multithreading*

SO e cenário atual

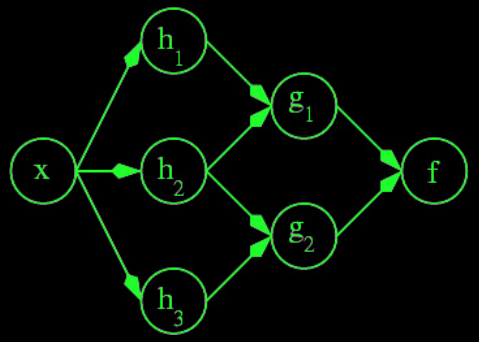
> Cenário comum: multiprogramação, tempo compartilhado, paralelismo real, *multithreading*

> Colaboração

SO e cenário atual

> Cenário comum: multiprogramação, tempo compartilhado, paralelismo real, *multithreading*

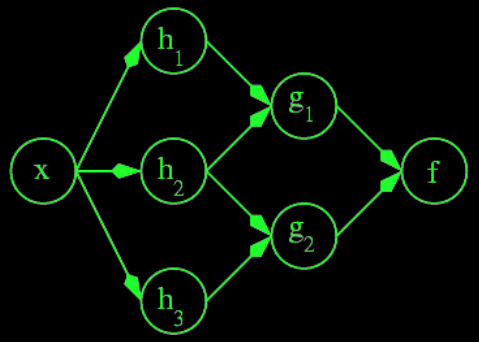
> Colaboração Dependência



SO e cenário atual

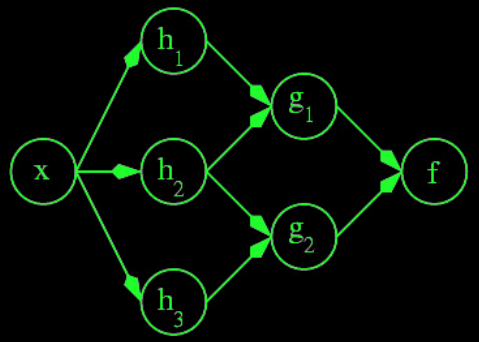
> Cenário comum: multiprogramação, tempo compartilhado, paralelismo real, *multithreading*

> Colaboração Dependência Concorrência



CIP: comunicação interprocessos

> Colaboração Dependência Concorrência



> Comunicação e sincronização de acesso de processos a recursos são fundamentais para evitar problemas
  > Inconsistência de dados
  > Dados usados em duplicidade

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

CONTA
saldo
100

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

CONTA
saldo
100

DEBITAR
saldo
100
Valor
35

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

CONTA
saldo
100

DEBITAR
saldo
65
Valor
35

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

CONTA
saldo
100

DEBITAR
saldo
65
Valor
35

CREDITAR
saldo
100
Valor
50

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

CONTA
saldo
100

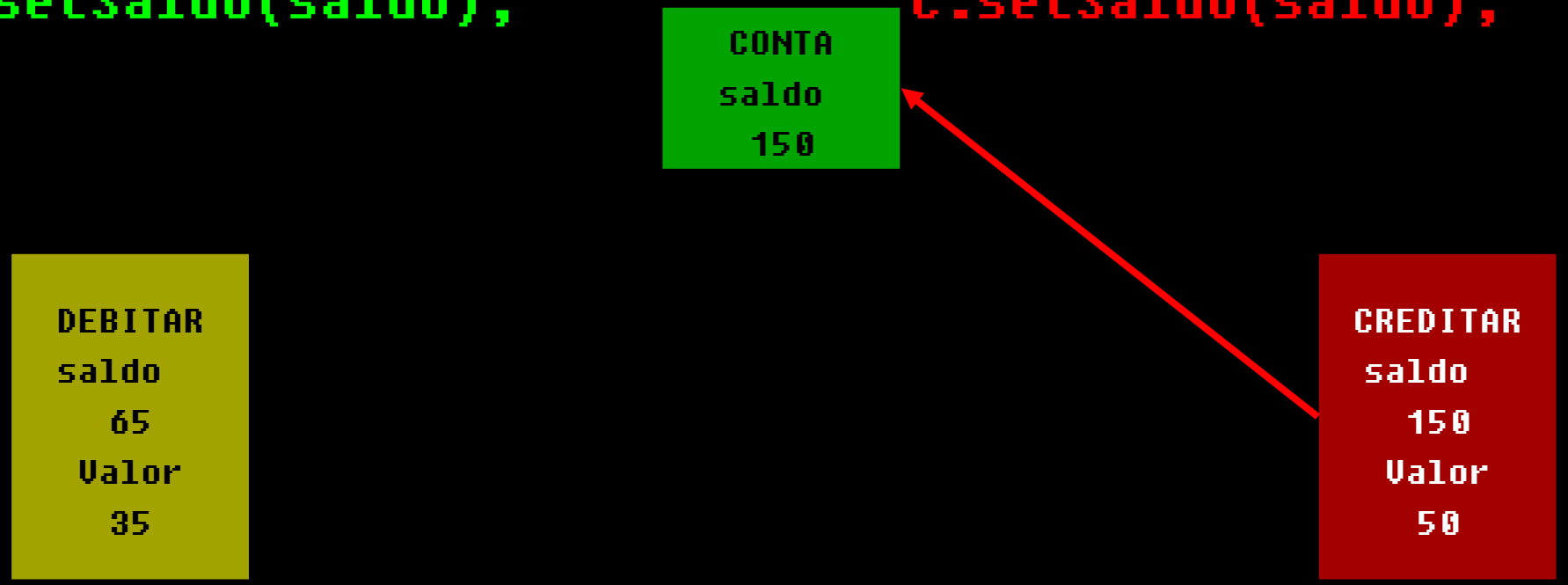
DEBITAR
saldo
65
Valor
35

CREDITAR
saldo
150
Valor
50

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```



CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

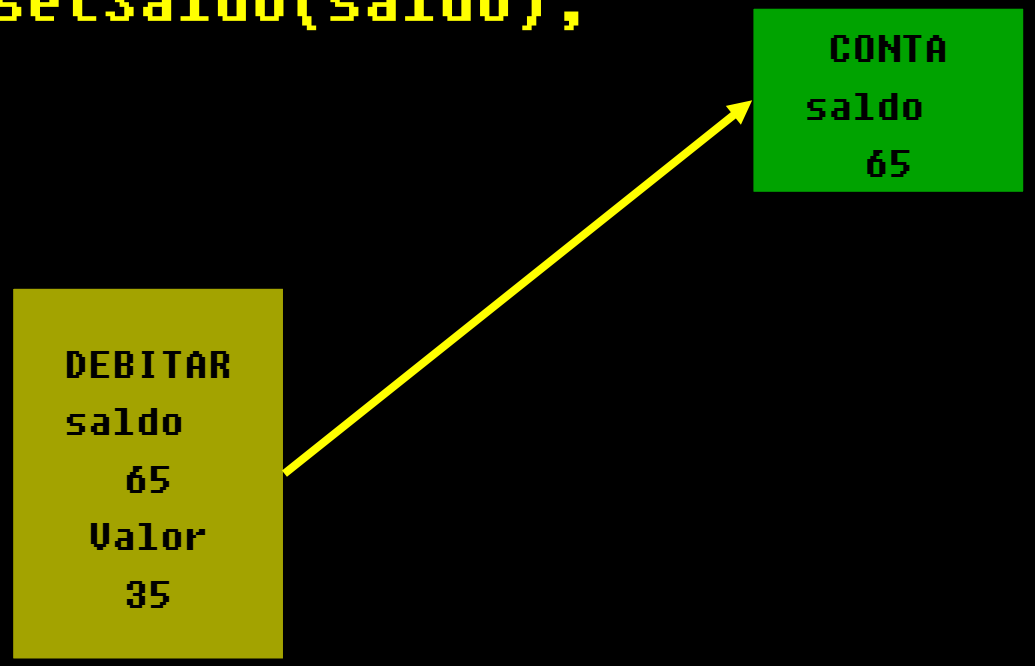
CONTA
saldo
150

DEBITAR
saldo
65
Valor
35

CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```



CIP: Comunicação interprocessos

```
Debitar(Conta c, valor v)
    saldo = c.saldo();
    saldo -=v;
    c.setSaldo(saldo);
```

```
Creditar(Conta c, valor v)
    saldo = c.saldo();
    saldo +=v;
    c.setSaldo(saldo);
```

Condição de corrida

> Quando dois ou mais processos compartilham recursos e o resultado final depende da ordem, velocidade ou momento de execução

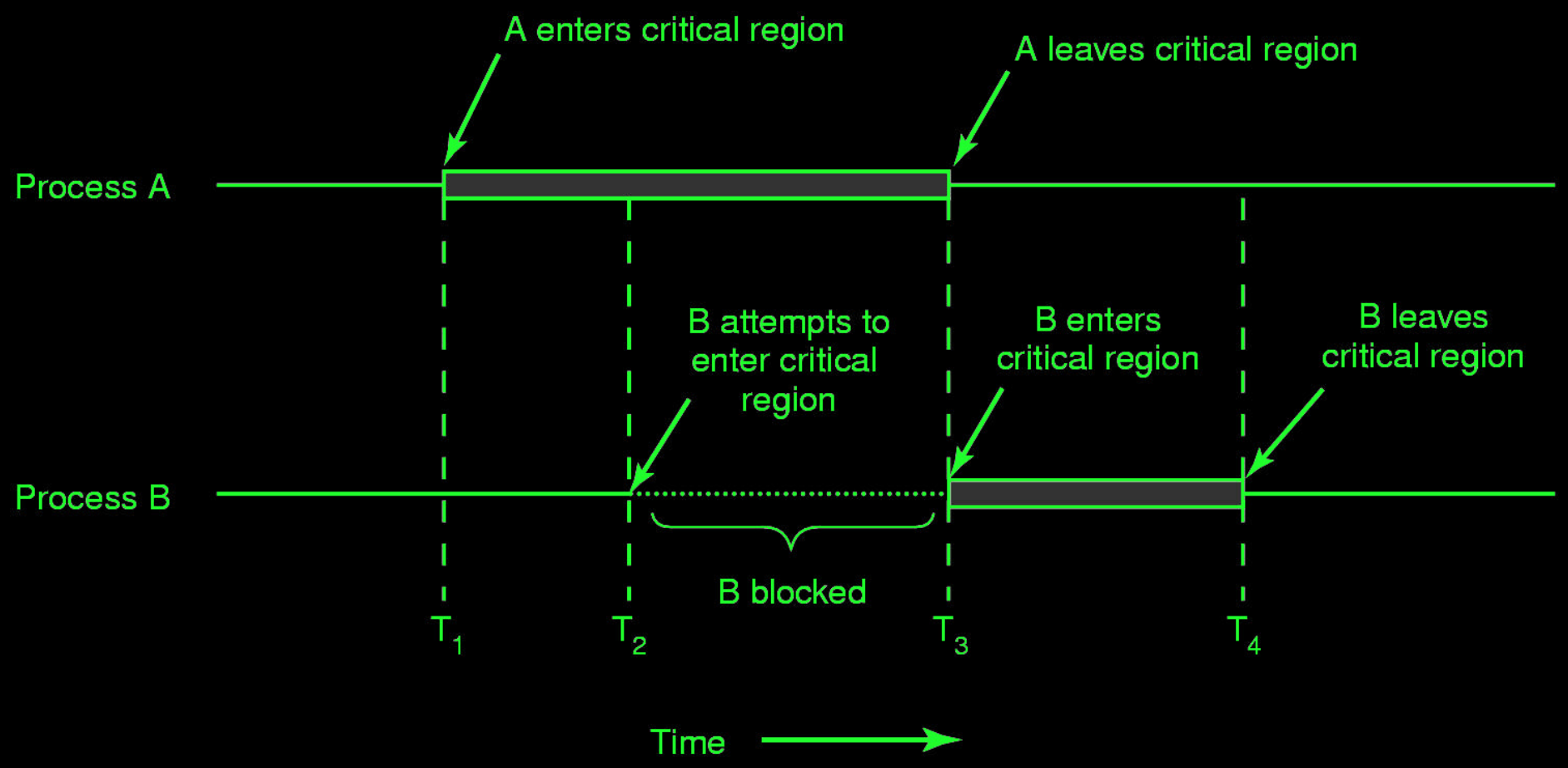
Região crítica

- Trecho(s) dos processos nos quais os dados compartilhados são acessados
  - Potencialmente perigosos

Exclusão mútua

> Mecanismo para garantir que, se um processo está em sua região crítica, outros processos serão impedidos de estar em suas regiões críticas

Regiões críticas e exclusão mútua



Fonte: TANENBAUM; BOS (2016)

Comunicação interprocessos

> Regiões críticas

—> Condições de corrida

—> Necessidade de exclusão mútua

Princípios da boa exclusão mútua

1. Somente um processo pode estar na região crítica a cada momento

2. Não se pode fazer suposições sobre velocidade e ordem de execução de processos

3. Nenhum processo fora da região crítica pode impedir outros processos de entrarem em suas regiões críticas

4. Um processo não pode esperar indefinidamente para entrar em sua região crítica

Princípios da boa exclusão mútua

1. Somente um processo pode estar na região crítica a cada momento

2. Não se pode fazer suposições sobre velocidade e ordem de execução de processos

OBRIGATORIAMENTE OU NÃO FUNCIONA

3. Nenhum processo fora da região crítica pode impedir outros processos de entrarem em suas regiões críticas

4. Um processo não pode esperar indefinidamente para entrar em sua região crítica

Princípios da boa exclusão mútua

1. Somente um processo pode estar na região crítica a cada momento

2. Não se pode fazer suposições sobre velocidade e ordem de execução de processos

3. Nenhum processo fora da região crítica pode impedir outros processos de entrarem em suas regiões críticas

4. Um processo não pode esperar indefinidamente para entrar em sua região crítica

DESEJADAMENTE OU TEREMOS ALGUM PROBLEMA